#D Sign the JWT using the JWSSigner object.
#E Convert the signed JWT into the JWS compact serialization.

To use the new token store, open the Main.java file in your editor and change the code that constructs the `JsonTokenStore` and `HmacTokenStore` to instead construct a `SignedJwtTokenStore`. You can reuse the same `macKey` that you loaded for the `HmacTokenStore`, as you're using the same algorithm for signing the JWTs. The code should look like the following, using the `MACSigner` and `MACVerifier` classes for signing and verification using HMAC:

```
var algorithm = JWSAlgorithm.HS256;    #A
var signer = new MACSigner((SecretKey) macKey);    #A
var verifier = new MACVerifier((SecretKey) macKey);    #A
TokenStore tokenStore = new SignedJwtTokenStore(    #B
        signer, verifier, algorithm,
"https://localhost:4567");    #B
var tokenController = new TokenController(tokenStore);
```

#A Construct the MACSigner and MACVerifier objects with the macKey
#B Pass the signer, verifier, algorithm, and audience to the SignedJwtTokenStore

You can now restart the API server, create a test user, and log in to see the created JWT:

```
$ curl -H 'Content-Type: application/json' \
  -d '{"username":"test","password":"password"}' \
  https://localhost:4567/users
{"username":"test"}
$ curl -H 'Content-Type: application/json' -u test:password \
```